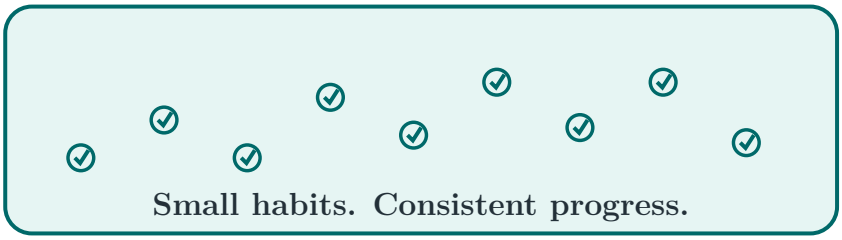
SOFTWARE PROJECT PROPOSAL

Ovvasjatra

A Simple Habit Tracker Application in Java



Project title:	Ovvasjatra (Habit Journey)
Language / UI:	Java, Java Swing
Process model:	Iterative and Incremental Model
Estimated duration:	8 weeks
Student:	Ishtiak Ahmed Ayon
Student ID :	IT24039
Department / University:	ICT / MBSTU
Course :	Software Engineering and Project Management
GitHub repository:	https://github.com/ishtiakahmedayon/SEPM-Capstone-Project

Contents

1 Executive Summary **2**

2 Problem Statement and Motivation **2**

3 Objectives and Scope **2**
3.1 Objectives . . . 2
3.2 In Scope . . . 2
3.3 Out of Scope for Version 1 . . . 2

4 Stakeholders and Users **2**

5 Requirements **2**
5.1 Functional Requirements . . . 3
5.2 Non-functional Requirements . . . 3

6 Software Process Model **3**

7 Project Schedule, Milestones, and Deliverables **3**
7.1 Gantt Chart . . . 4

8 System Design Diagrams **4**
8.1 Data Flow Diagram (Level 0) . . . 4
8.2 Use Case Diagram . . . 4
8.3 Activity Diagram: Logging a Habit . . . 5
8.4 Entity Relationship Diagram (ERD) . . . 5
8.5 Class Diagram . . . 6
8.6 Object Diagram (Illustrative Snapshot) . . . 6

9 Prototype Interface **6**

10 Implementation and Technology **7**

11 Testing Plan and Acceptance Criteria **7**

12 Risks and Mitigation **7**

13 GitHub and Submission Checklist **7**

14 Conclusion **8**

1 Executive Summary

Ovvasjatra (, meaning “habit journey”) is a lightweight Java desktop application that helps a user build consistent routines. A user can create habits, set a frequency or daily target, record completion for a date, and review a simple progress summary. The initial version focuses on a clear interface and reliable local storage rather than social features or cloud accounts.

2 Problem Statement and Motivation

People often begin routines but struggle to track consistency over time. Paper notes and generic to-do lists do not always provide a dedicated view of recurring habits. Ovvasjatra will provide a small, focused tool to record habits and see completion progress. The project is suitable for an individual software engineering project because it includes requirements analysis, design diagrams, object-oriented implementation, testing, and documentation.

3 Objectives and Scope

3.1 Objectives

- Create, view, edit, and archive personal habits.
- Record whether a habit is completed on a selected day.
- Show a daily checklist and basic completion statistics.
- Store data locally so it remains available after restarting the app.
- Apply object-oriented design and test the core use cases.

3.2 In Scope

Single-user desktop use; habit name and description; daily/weekly frequency; completion records; simple streak/percentage summary; local persistence; input validation.

3.3 Out of Scope for Version 1

Cloud synchronisation, multiple accounts, social sharing, mobile apps, push notifications, AI recommendations, and integrations with wearable devices.

4 Stakeholders and Users

Stakeholder	Interest / responsibility
Primary user	Creates habits, logs completion, reviews progress.
Student developer	Analyses requirements, designs, implements, tests, and documents the app.
Course instructor / supervisor	Reviews proposal, milestones, diagrams, and final demonstration.

5 Requirements

5.1 Functional Requirements

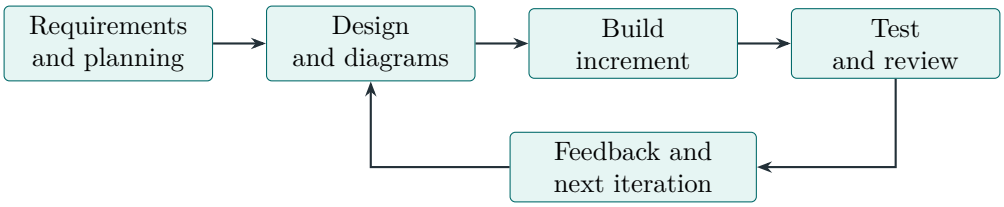
ID	Requirement
FR-01	The user can add a habit with a name, description, and frequency.
FR-02	The user can view all active habits in a list.
FR-03	The user can edit or archive a habit.
FR-04	The user can mark a habit complete or undo completion for a date.
FR-05	The app displays completed/total habits and a completion percentage for the selected day.
FR-06	Habit details and completion records persist locally between launches.
FR-07	The app validates required fields and gives understandable error messages.

5.2 Non-functional Requirements

- Usability: common actions should be understandable without training.
- Reliability: completion records should not be lost during normal application exit.
- Performance: display a typical personal habit list in under one second on a standard student PC.
- Maintainability: separate UI, business logic, and persistence responsibilities.
- Privacy: version 1 stores data locally and does not transmit personal data.
- Compatibility: Java 17 or later with a desktop environment supporting Swing.

6 Software Process Model

The project will use the **Iterative and Incremental Model**. Each iteration delivers a small working increment, receives feedback, and refines the next increment. This is appropriate for a small student project because the interface and requirements can be improved after early demonstrations.

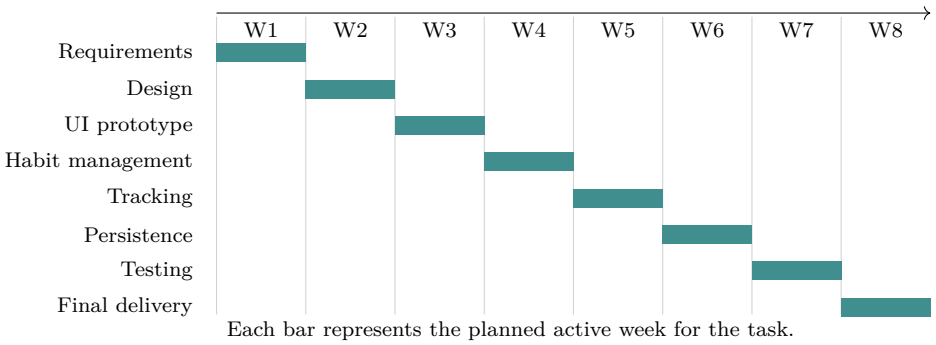


7 Project Schedule, Milestones, and Deliverables

The schedule is an estimate for an eight-week individual project. Weeks may be adjusted with supervisor feedback.

Week	Activity	Main output	Milestone
1	Idea and requirements	Proposal draft, requirements list	M1: Scope approved
2	Analysis and design	DFD, use case, activity, ERD, UML drafts	M2: Design reviewed
3	UI prototype	Swing screens and navigation	M3: UI demo
4	Habit management	Add, list, edit, archive habits	M4: Habit CRUD
5	Completion tracking	Daily completion and summary logic	M5: Tracking works
6	Persistence	Local storage and validation	M6: Data persists
7	Testing and refinement	Test cases, bug fixes, usability pass	M7: Test pass
8	Final documentation	Final PDF, source code, README, demo	M8: Submission

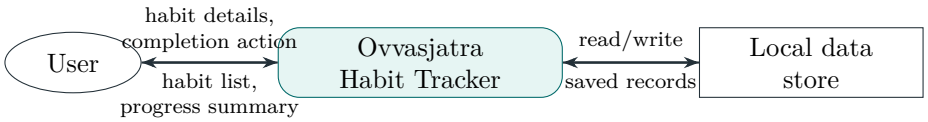
7.1 Gantt Chart



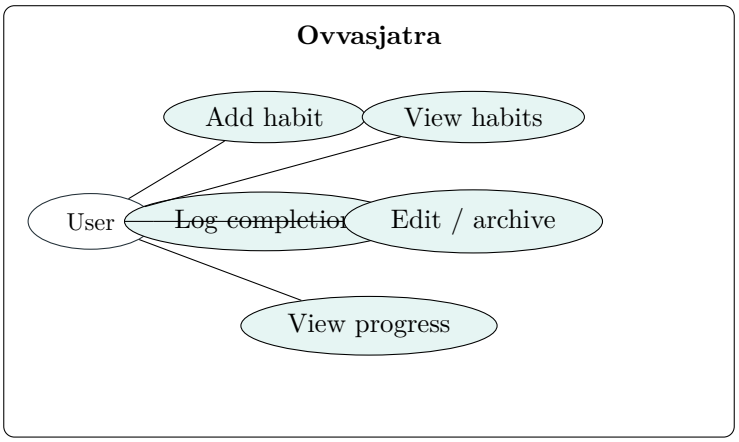
Each bar represents the planned active week for the task.

8 System Design Diagrams

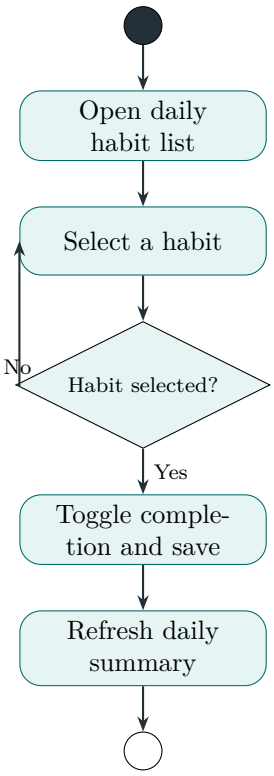
8.1 Data Flow Diagram (Level 0)



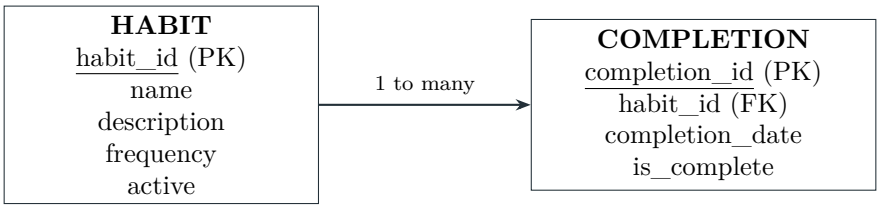
8.2 Use Case Diagram



8.3 Activity Diagram: Logging a Habit

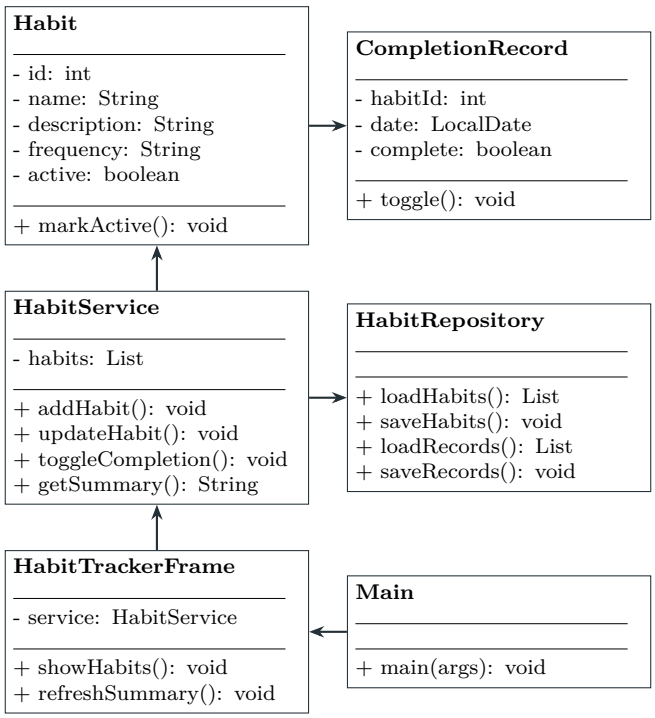


8.4 Entity Relationship Diagram (ERD)



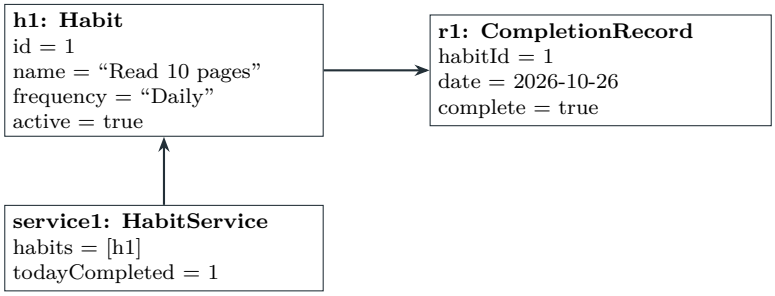
One habit can have many dated completion records. The combination of habit and date should be unique to prevent duplicate logs.

8.5 Class Diagram



The class diagram defines six core classes. The first prototype will implement these responsibilities with a simple Swing interface and file-based persistence.

8.6 Object Diagram (Illustrative Snapshot)



This is an example runtime snapshot, not actual user data.

9 Prototype Interface

The first interface will be implemented in Java Swing. It will include a habit name field, frequency selector, add button, habit table, completion action, and a progress summary. The prototype uses at least five classes from the class diagram: `Main`, `Habit`, `CompletionRecord`, `HabitService`, `HabitRepository`, and `HabitTrackerFrame`. Source code is provided with this proposal.

10 Implementation and Technology

Component	Choice / purpose
Language	Java 17+; object-oriented implementation.
GUI	Java Swing; desktop interface without external UI dependencies.
Persistence	Local serialized data file in the user's home directory (prototype); database may be considered in a later version.
Design	UI / service / repository separation.
Version control	Git and GitHub; commits grouped by feature or milestone.
Testing	Manual use-case tests plus unit tests for habit and summary logic where practical.

11 Testing Plan and Acceptance Criteria

Test ID	Test case	Expected result
T-01	Add a habit with a valid name	Habit appears in the list.
T-02	Submit an empty name	Validation message appears; no habit is added.
T-03	Mark a habit complete	Daily completion count and percentage update.
T-04	Toggle the same habit again	Completion is undone without duplicate records.
T-05	Close and reopen the app	Saved habits and logs remain available.
T-06	Archive a habit	Habit is no longer shown as active; old records remain.
T-07	Enter unusual characters in a name	App remains stable and displays the name safely.

Acceptance: all critical test cases pass, the app runs from a documented command, and the source and proposal are available in the GitHub repository.

12 Risks and Mitigation

Risk	Mitigation
Scope grows beyond the semester	Keep cloud, mobile, and social features out of version 1.
Persistence bugs	Test save/load repeatedly and keep repository logic isolated.
Limited development time	Deliver a working increment each week; prioritise core tracking.
UI and logic become coupled	Keep UI event handlers thin and use HabitService for rules.

13 GitHub and Submission Checklist

- **Repository:** https://github.com/[username]/ovvasjatra (replace placeholder with the actual repository URL).
- Include this proposal PDF and its LaTeX source.
- Include Java source files, README with run instructions, and test checklist.
- Add screenshots of the running prototype and meaningful Git commits.
- Replace the cover-page placeholders before submission.

14 Conclusion

Ovvasjatra is a focused, achievable Java project that addresses habit consistency through habit management, daily completion records, and progress summaries. The iterative plan, requirements, six requested diagrams, schedule, testing plan, and Swing prototype provide a practical foundation for implementation and final demonstration.